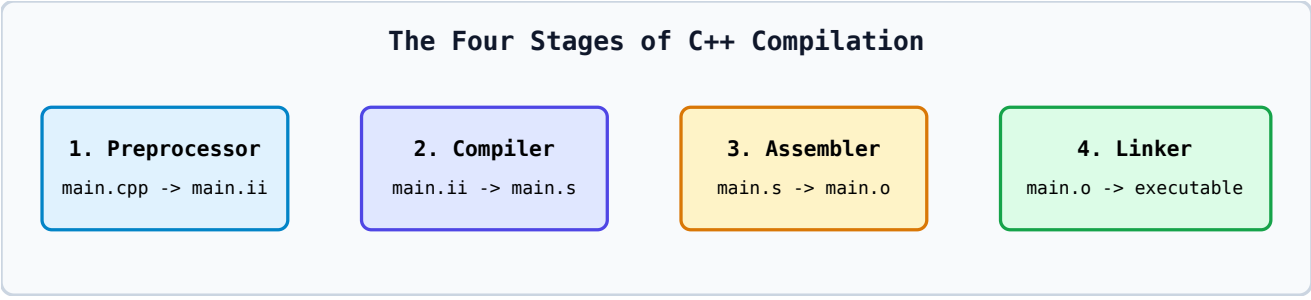
01 CPP FUNDAMENTALS • THEORY

01 compilation pipeline and program structure

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

01 — The Four Stages of C++ Compilation and Program Structure



C++ Compilation Pipeline

1. The Four Stages of Turning `.cpp` into an Executable

1. **Preprocessing (`g++ -E`)**: Expands `#include <iostream>`, `#define` macros, and strips comments, producing a translation unit (`.i` / `.ii`).
2. **Compilation (`g++ -S`)**: Parses C++ syntax, checks types and access rules, optimizes, and generates architecture-specific assembly code (`.s`).
3. **Assembly (`g++ -c`)**: Translates assembly instructions into machine-code object files (`.o` / `.obj`).
4. **Linking (`ld`)**: Resolves external symbols across object files and standard library binaries (`libstdc++`) to produce the final executable binary.

2. Why `int main()` is Mandatory in Standard C++ (`[basic.start.main]`)

The ISO C++ standard mandates that the return type of `main` in a hosted environment **must be** `int`.

- Writing `void main()` was a non-standard extension in 1990s MS-DOS compilers (like Turbo C++) and is **ill-formed (Compilation Error)** in modern standard C++ (`g++ -pedantic-errors`).
- If control reaches the end of `int main()` without encountering a `return` statement, the compiler implicitly inserts `return 0;` (indicating successful execution to the operating system).